Sentiment Analysis of Code-Mixed Hinglish Tweets

CS60075 NLP Assignment II · Project 6 · SentiMix Partition B

Group 17: Yashvardhan Jain (23CS10080) · Nishant (23EE10044) · Devubarki Pushyamithra (24CS10092)

The problem: Hindi and English in one tweet

“yeh movie bahut acchi thi, must watch!”

No standard spelling
bahut / bhut / bohot are the same word

Same string, two languages
main, par, to can be Hindi or English

Noisy social-media text
split URLs, @mentions, hashtags, emojis

4,928 / 616 / 616

train / validation / test tweets
Partition B of SemEval-2020 SentiMix

93%

of tweets mix both languages
the task is genuinely code-mixed

Three systems

1

Classical pipeline

Language ID (char 1–4-gram TF-IDF + logistic regression) → Dakshina spelling normalization → word + char TF-IDF + Random Forest

2

MuRIL fine-tuned on raw tweets

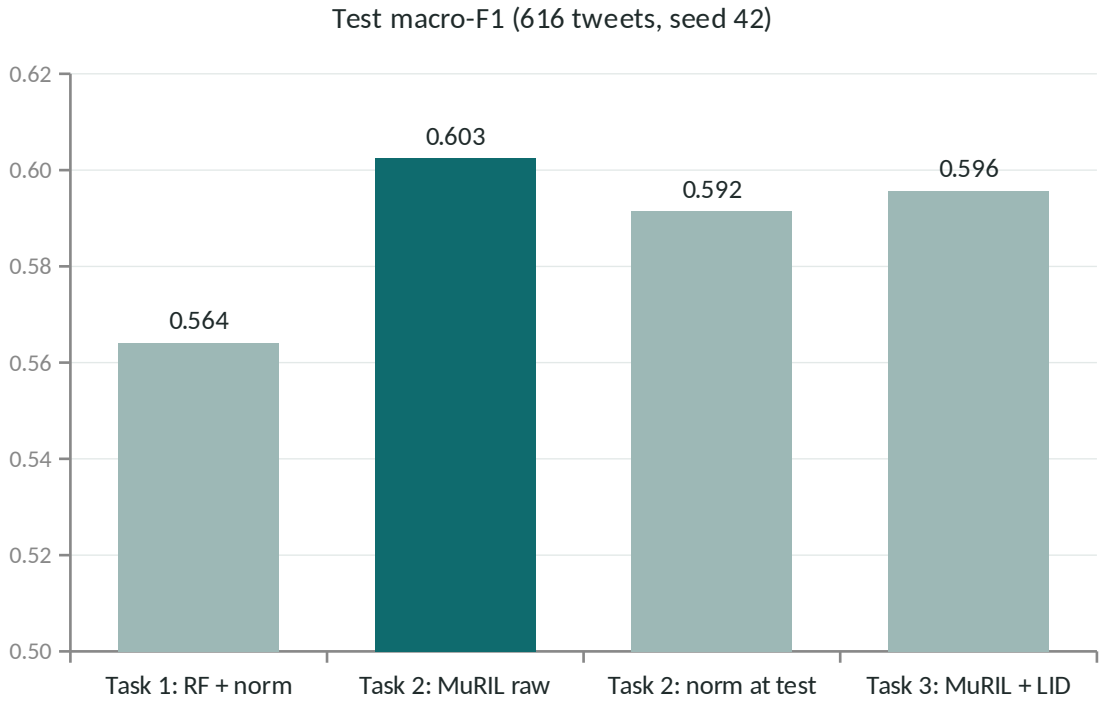
google/muril-base-cased, 4 epochs, lr 2e-5; also tested with normalized input at test time only

3

MuRIL + language tags

Per-token 16-dim tag embedding concatenated to each word embedding; trained on normalized text

Results: MuRIL is the clear step up



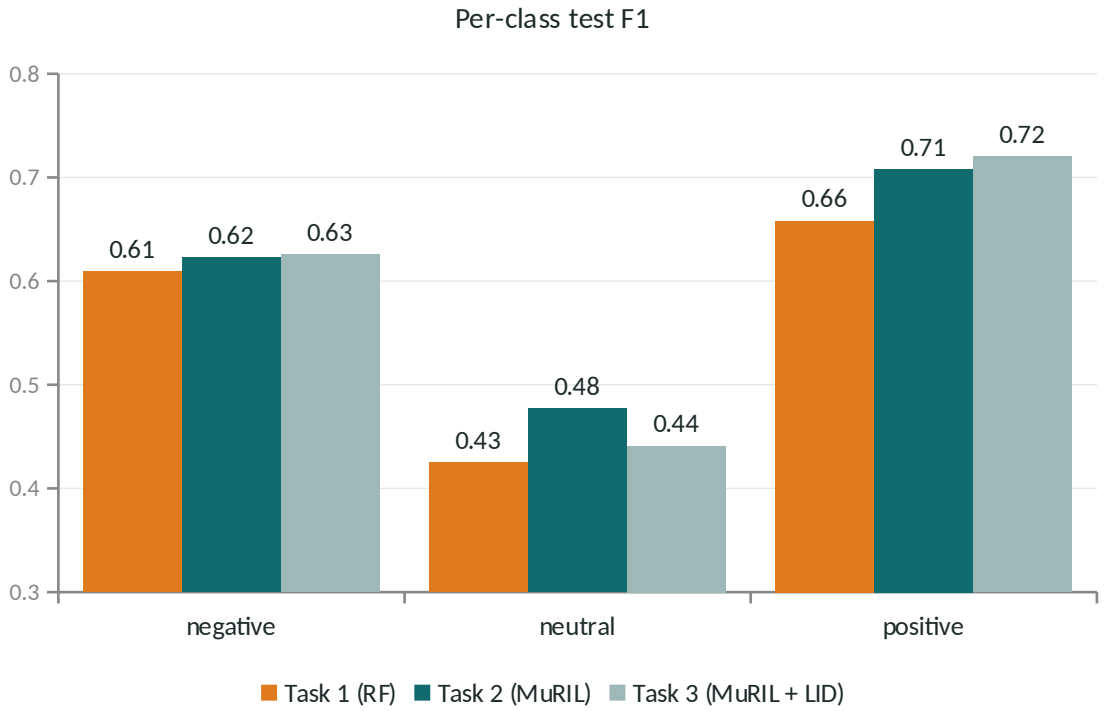
0.603

best: MuRIL on raw text
test accuracy 0.606

+3.8 pts

over the classical pipeline
RF: 0.564 macro-F1, 0.570 accuracy

Comparing the tasks: per-class F1



MuRIL wins in every class
word order, negation and transliterated Hindi, which a bag of n-grams cannot see

The RF over-predicts negative
negative recall 0.78 at precision 0.50 (balanced class weights)

Neutral is the hardest class
for all three models

Task 2 vs Task 3: is the gap real?

p = 0.85

exact McNemar test
58 tweets only Task 2 gets right, 55 only Task 3

0.7 pts

Task 3 below Task 2
0.596 vs 0.603 test macro-F1

616

test tweets
one tweet = 0.16 accuracy points

Each model was trained once (seed 42) on 4,928 tweets, so differences below about 1–2 points are within run-to-run variation. Task 2 and Task 3 are statistically indistinguishable.

Why normalization does not help

The dictionary ignores context. It rewrites **8.6%** of test tokens (1,042 of 12,120, in 458 of 616 tweets), and some rewrites change the meaning:

`ji → ge`	honorific becomes a meaningless string (57 times)
`baad → bad`	“after” becomes the English word “bad”
`har → haar`	“every” becomes “defeat”
`main → man`	the pronoun “I” becomes “mind”

MuRIL trained on raw text: normalizing only its test input costs 1.1 points (0.603 → 0.592).

Even correct rewrites (nahi → nahin) move the text away from the raw spellings MuRIL was fine-tuned on.

Task 3: why language tags did not add accuracy

1. **Normalization cancels any gain**
Task 3 trains on normalized text, which costs MuRIL about 1 point

2. **The tags are noisy and redundant**
13% tag errors (LID accuracy 0.867); MuRIL's subwords already signal the language

3. **Small data, single run**
0.7-point gap, p = 0.85

Design note

A fully learned 784→768 projection collapsed to predicting only “neutral” (F1 0.188).

Fix: keep the word block at the identity and learn only the tag part, starting at zero:

$$e' = e + W \cdot z$$

Conclusions

- MuRIL is the best single model (0.603), 3.8 points above the classical pipeline
- Context-free normalization hurts a model trained on raw text (0.603 → 0.592)
- Language tags gave no significant gain on top of MuRIL ($p = 0.85$)
- Next: tags on raw text, a dictionary that skips ambiguous romanizations, several seeds per model

Individual contributions

Yashvardhan Jain

23CS10080

Data preprocessing

SemEval parsing, URL and mention cleaning, Partition B split, Dakshina dictionary

Task 2: MuRIL fine-tuning

training on raw tweets, raw vs normalized test evaluation

Nishant

23EE10044

Task 1: classical pipeline

char n-gram language-ID tagger, Dakshina normalization, TF-IDF + Random Forest

Task 3: LID-augmented MuRIL

per-token tag embedding, [I | W_lid] projection fix